Code Coverage Report (Mini-Project Part-3 §2.b.iii)

Course: Software Engineering | **Section:** A | **Team Number:** 9
Target System: Online Exam Portal (OEP)
Testing Window: 2 – 6 November 2026 (Sprint 3); results regenerated 12 November 2026 after the v1.0.1 fixes
Team: Aadish Sarin (`PES2UG24CS007` , QA Lead) · A S Vincent (`PES2UG24CS002` , Dev Lead) · Aditya Rao (`PES2UG24CS034` , DevOps Lead) · Akshansh Agrawal (`PES2UG24CS045` , Performance Lead)
Code tested: commit `57e01fd` on `main` · **Repository:** https://github.com/vinnieee7/SE-Online-Exam-Portal-Team09

1. Summary

Coverage was measured with Jest's built-in Istanbul instrumentation while running **all 85 automated tests** (5 unit suites + 3 integration suites) against the backend in `backend/src`.

Metric	Covered / total	Coverage
Statements	602 / 796	**75.62%**
Lines	574 / 758	**75.72%**
Functions	61 / 77	**79.22%**
Branches	247 / 413	**59.80%** (see Branch Coverage Report)

Command: `cd backend && npx jest --runInBand --coverage` — run 12 November 2026, commit `57e01fd`, 85/85 tests passed. The full interactive HTML report is in `docs/coverage/index.html`.

2. Coverage by file

File	Role	Stmts %	Branch %	Funcs %	Lines %
`src/app.js`	Express app setup and route mounting	84.09	62.50	80.00	86.11
`src/secureServer.js`	HTTPS server (TLS 1.2+)	83.33	100.00	50.00	83.33
`src/config/db.js`	SQLite connection and query helpers	84.84	61.53	100.00	93.33
`src/controllers/authController.js`	Register, login, password reset, user admin	65.09	61.70	57.14	65.09
`src/controllers/deliveryController.js`	Start attempt, timer, auto-save, submit	72.80	53.84	85.71	72.56
`src/controllers/evalController.js`	Grading, scorecards, analytics	78.67	64.28	85.71	77.77

File	Role	Stmts %	Branch %	Funcs %	Lines %
`src/controllers/examController.js`	Exams, question bank, shuffle	58.19	52.66	75.00	57.83
`src/db/seed.js`	Database schema and seed data	95.29	90.00	60.00	95.00
`src/middleware/auth.js`	JWT check and role guard	92.59	93.75	100.00	92.59
`src/middleware/logger.js`	Audit log writer	83.33	66.66	100.00	83.33
`src/routes/auditRoutes.js`	Audit log route	91.66	100.00	100.00	91.66
`src/routes/authRoutes.js`	Auth routes	100.00	100.00	100.00	100.00
`src/routes/deliveryRoutes.js`	Delivery routes	100.00	100.00	100.00	100.00
`src/routes/evalRoutes.js`	Evaluation routes	100.00	100.00	100.00	100.00
`src/routes/examRoutes.js`	Exam routes	100.00	100.00	100.00	100.00
`src/utils/serverError.js`	Generic 500 response helper	33.33	100.00	0.00	33.33
All files		**75.62**	**59.80**	**79.22**	**75.72**

The React frontend is not instrumented; it was tested in the browser (`TC-Take-06`) and by the accessibility audit.

3. What each test layer adds

Measured by running Jest with coverage on each subset of suites:

Suites run	Tests	Statements	Branches
Unit suites only	34	19.51%	5.23%
Unit + `api.test.js` (the Sprint 3 baseline)	53	69.11%	48.42%
All suites (unit + api + security + verification)	**85**	**75.62%**	**59.80%**

The unit suites mostly test rules in isolation, so most coverage comes from the integration suites, which drive the real controllers over HTTP. The security and verification suites added in v1.0.1 raised statement coverage by 6.5 points and branch coverage by 11.4 points.

4. Code that is not covered, and why

Where	Not covered	Reason / plan
`examController.js` `updateExam`, `deleteExam`, `deleteQuestion`	Editing/publishing and deleting exams and questions	Works in the Instructor screen (shown in the demo) but has **no automated test yet** — the largest gap
`authController.js` `updateUserStatus`	Admin changing a role or locking a user through the API	`TC-Sec-03f` locks the account directly in the database; an API-level test is still to be added
`evalController.js` `getMyScorecard`	Student opening a scorecard by exam id	Used by the dashboard; covered only through the browser

Where	Not covered	Reason / plan
Validation branches in all controllers	Missing-field 400s, "exam not started / already ended", "not published"	Only the main rejections (end before start, IDOR, unknown question id) are tested
`catch` blocks → `serverError.js`	Database failure responses	Need a forced database error; not simulated
`deliveryController.js` lines 247–282, `app.js` lines 47–51	Fallback code from before the modules were merged	Unreachable now that all modules are present; can be deleted

5. Conclusion

Backend statement coverage is **75.62%** and every route file is at 100%. All requirement-critical paths named in the RTM have an executed test. The gaps above are the next tests to write; none of them hides a known defect.

Prepared by Aadish Sarin (QA Lead).